← Вернуться к блогу

Ядро WordPress Критическая уязвимость RCE CVE-2026-63030

CVE-2026-60137

Исследование BugBunny.ai • 18 июля 2026 г. • 20 мин. чтения

Как две ошибки в ядре WordPress привели к RCE без аутентификации

Как путаница с пакетными маршрутами REST, скалярная SQL-инъекция, кэш объектов WordPress, Customizer и вложенная диспетчеризация REST сформировали путь к выполнению кода без аутентификации — и как на это реагировать.

ДЕЙСТВИЕ DEFENDER

Установите исправление для ядра WordPress. Проверьте установленную версию и контрольные суммы, даже если включены автоматические обновления. В поддерживаемых ветках исправления были внесены в версии 7.0.2, 6.9.5 и 6.8.6.

ГРАНИЦА ДОКАЗАТЕЛЬСТВ

ПОДТВЕРЖДЕНО ПОСТАВЩИКОМ

Путаница с маршрутами в сочетании с SQL-инъекцией приводит к удаленному выполнению кода.

BUGBUNNY-REPRODUCED

Один стандартный путь был восстановлен в изолированной лаборатории WordPress 7.0.1.

УМЫШЛЕННО ОПУЩЕНО

Исполняемые эксплойты, учетные данные и произвольные команды.

ON THIS PAGE

Большинство историй о дистанционном выполнении кода начинаются с чего-то опасного: `eval()` , небезопасной десериализации или неограниченной загрузки

файлов.

Эта история начинается с бухгалтерии.

Пакетный обработчик REST в WordPress поддерживал несколько параллельных массивов. Некорректный запрос был записан в двух из них, но не в третьем. С этого момента корректный запрос мог быть связан с обработчиком маршрута, предназначенным для другого запроса.

Это несоответствие не было оболочкой. Как и отдельная SQL-инъекция, которую оно открывало. Но когда эти ошибки встретились с кэшированием постов в WordPress, логикой обновления oEmbed, восстановлением иерархии, состоянием Customizer и глобально общим пользовательским контекстом, результатом стал путь к созданию администратора без аутентификации. Последний шаг к выполнению PHP был почти скучным: обычно администраторам разрешено устанавливать плагины.

WordPress выпустил **версию 7.0.2 17 июля 2026 года**, в описании проблемы говорится о путанице с пакетными маршрутами REST API в сочетании с SQL-инъекцией, которая приводила к удаленному выполнению кода. Из-за серьезности проблемы WordPress включил принудительное автоматическое обновление для уязвимых сайтов. Администраторам все равно следует проверить, действительно ли обновление установлено. (Объявление о выпуске WordPress 7.0.2, рекомендация по CVE-2026-63030)

60-секундная версия

Атака — это цепочка, а не одна катастрофическая функция:

КРАТКИЙ ОБЗОР ЦЕПОЧКИ АТАК

Ни один из этапов не приводит к выполнению кода. Проблема возникает, когда данные пересекают пять границ доверия.

1

ЗАПИСЬ

Анонимный пакетный запрос REST

Некорректно сформированный элемент попадает в WordPress через общедоступную конечную точку пакетной обработки.

2

НЕСООТВЕТСТВИЕ

Запросы и обработчики расходятся

Отсутствие элемента в массиве нарушает соответствие маршрута, разрешений и обратного вызова.

3

ГРАНИЦА ДАННЫХ

Строки SQL становятся доверенными кэшированными объектами

Скаляр достигает WP_Query, после чего зараженные объекты WP_Post попадают в кэш запросов.

4

PRIVILEGE TRANSITION

Customizer identity overlaps REST re-entry

A temporary administrator switch remains active during a nested REST dispatch.

5

IMPACT

Administrator creation leads to PHP execution

The protected users endpoint creates an admin; normal plugin installation runs code.

No vulnerable plugin is required. No password needs to be cracked. No MySQL `FILE` privilege or `INTO OUTFILE` trick is necessary.

Who is affected?

The two CVEs overlap, but they do not affect every branch in exactly the same way. (Official release matrix)

WordPress branch	Impact	Fixed release
7.0.0–7.0.1	SQL injection and the critical REST-to-RCE chain	7.0.2
6.9.0–6.9.4	SQL injection and the critical REST-to-RCE chain	6.9.5
6.8.0–6.8.5	Standalone SQL injection; not the full REST-to-RCE chain	6.8.6
Earlier than 6.8	Not affected by these two specific issues	Not applicable
7.1 beta	Both issues	7.1 beta 2

The SQL injection is tracked as **CVE-2026-60137 / GHSA-fpp7-x2x2-2mjf**. The combined pre-authentication RCE is **CVE-2026-63030 / GHSA-ff9f-jf42-662q**. (SQLi advisory, RCE advisory)

Bug one: the batch handler's three ledgers drift apart

`WP_REST_Server::serve_batch_request_v1()` effectively kept three ledgers:

```
SEQUENCE                                                    Copy
```

```
$requests    parsed request objects—or parsing errors
$matches     route and handler tuples
$validation  validation results
```

In WordPress 7.0.1, a malformed path became a `WP_Error` in `$requests`. During the next loop, the error was added to `$validation`, but the code continued without adding a placeholder to `$matches`.

Later, execution walked `$requests` and looked up `$matches[$i]` using the same index:

SEQUENCE Copy

```
requests:   [ ERROR, request A, request B ]
matches:    [ handler A, handler B       ]
                         ↑
              request A receives handler B
```

This is a classic **parallel-array desynchronization**—a zipper with one missing tooth.

The affected logic can be reduced to this:

PHP Copy

```php
// WordPress 7.0.1 — simplified.
foreach ( $requests as $single_request ) {
    if ( is_wp_error( $single_request ) ) {
        $validation[] = $single_request;
        continue; // No corresponding $matches entry.
    }

    $matches[] = $this->match_request_to_handler( $single_request );
}

foreach ( $requests as $i => $single_request ) {
    $match = $matches[ $i ];
```

```
    // Request i may now execute with a later request's handler.
}
```

The 7.0.2 batch fix is only one line:

PHP Copy

```php
if ( is_wp_error( $single_request ) ) {
    $has_error     = true;
    $matches[]     = $single_request; // Preserve index alignment.
    $validation[] = $single_request;
    continue;
}
```

One important nuance: this is not simply “a public permission check paired with a privileged callback.” The entire handler tuple shifts, including its permission callback and execution callback. The primitive is more subtle: a request object prepared for one route is consumed under another route's handler and schema assumptions. (Vulnerable 7.0.1 batch implementation)

Bug two: an ID parameter reaches SQL without universal normalization

The posts REST controller exposes `author_exclude` and maps it internally to `WP_Query`'s `author__not_in` variable. Under normal routing, the REST schema expects an array of integers. Route confusion lets the posts handler receive a request that was not sanitized against that schema. (REST parameter mapping)

In 7.0.1, `WP_Query` normalized the value only if it was already an array:

PHP Copy

```php
// WordPress 7.0.1 — abbreviated.
if ( ! empty( $query_vars['author__not_in'] ) ) {
    if ( is_array( $query_vars['author__not_in'] ) ) {
        $query_vars['author__not_in'] = array_map(
            'absint',
            $query_vars['author__not_in']
        );
    }

    $ids = implode( ',', (array) $query_vars['author__not_in'] );
    $where .= " AND posts.post_author NOT IN ($ids) ";
}
```

An attacker-controlled scalar skipped the `absint()` branch, was cast to an array only at `implode()`, and entered the SQL fragment.

WordPress 7.0.2 normalizes every accepted shape first:

PHP Copy

```php
$ids = wp_parse_id_list( $query_vars['author__not_in'] );

if ( $ids ) {
    sort( $ids );
    $where .= sprintf(
        ' AND posts.post_author NOT IN (%s) ',
        implode( ',', $ids )
    );
}
```

The query hardening patch carries a lesson that applies well beyond WordPress: a parameter named “ID” is not safe because callers are expected to send numbers. Normalize it at the last trust boundary before SQL construction.

SQL injection is not the same thing as a shell

At this point the attacker can influence a `SELECT`, but MySQL is not executing operating-system commands. So how does the chain cross from query manipulation into application control?

The answer is **object hydration**.

`WP_Query` turns database result rows into `WP_Post` objects and primes WordPress's request-local object cache. A crafted query result can therefore behave like a post that exists for the rest of that PHP request—even when its fields did not come from a genuine row in the database. (WordPress 7.0.1 query result handling, post-cache priming)

That creates an **in-memory `WP_Post` cache-poisoning primitive**. It is not yet a durable write, so the next problem is finding legitimate core code that reads the poisoned object and writes it back.

oEmbed becomes the write bridge

WordPress's oEmbed cache supplies that bridge.

The exploit uses two top-level anonymous requests:

1. The first request renders synthetic ID-zero posts containing valid same-site embeds. `WP_Embed` persists genuine `oembed_cache` rows.
2. The second request returns forged post objects using the IDs of those real cache rows. Those objects populate the new request's ordinary, non-persistent `posts` cache.

3. A new embed render asks WordPress to refresh one of the genuine cache rows. The database lookup finds the real ID, but `get_post()` returns the poisoned object from memory.

There is a tiny PHP detail at the center of this step: the forged cached content is the string `'0'`. Humans see a value. PHP's `empty('0')` evaluates to true.

The oEmbed logic, simplified, looks like this:

PHP Copy

```php
$cached_post_id = $this->find_oembed_post_id( $key );
$cached_post    = get_post( $cached_post_id ); // May come from object cache.
$cache          = $cached_post->post_content;

if ( ! empty( $cache ) ) {
    return $cache;
}

$html = wp_oembed_get( $url );

wp_update_post( array(
    'ID'           => $cached_post_id,
    'post_content' => $html,
) );
```

That sparse `wp_update_post()` call is the bridge. It supplies only the ID and new content. `wp_update_post()` first loads every "original" field with `get_post()`, merges the sparse update over that object, and writes the result. If the object cache supplied forged original fields, legitimate core code persists them. (oEmbed update path, `wp_update_post()` merge behavior)

This is the pivotal move: **data returned by SQL becomes trusted application state, and trusted application state becomes a real database update.**

Two small parent cycles turn data into control flow

WordPress tries to protect hierarchical posts from impossible parent relationships. When it detects a cycle, `wp_check_post_hierarchy_for_loops()` repairs the loop by calling `wp_update_post()` on each affected member.

That normally helpful behavior becomes a recursion gadget when the “original” posts are poisoned in memory.

Our reproduced chain uses two tiny graphs:

```
SEQUENCE                                                    Copy

oEmbed refresh → A → B ↔ C

Customizer save → D → E ↔ F
```

The first repair reaches **B**, a poisoned object shaped as a past-due `future customize_changeset`. Because its scheduled date is already in the past, WordPress normalizes `future` to `publish`. That produces a real status transition and invokes `_wp_customize_publish_changeset()`. (Hierarchy repair, future-to-publish normalization, Customizer publish callback)

The crucial point is not the letters or IDs. It is that a safety mechanism performs additional sparse updates, and every sparse update reuses attacker-influenced cached fields.

The Customizer temporarily becomes an administrator—by design

Customizer changesets remember which user originally saved each setting. During publication, WordPress temporarily switches to that stored identity before saving the setting:

PHP Copy

```php
$original_user_id = get_current_user_id();

foreach ( $setting_ids as $setting_id ) {
    if ( isset( $setting_user_ids[ $setting_id ] ) ) {
        wp_set_current_user( $setting_user_ids[ $setting_id ] );
    }

    $setting->save();
}

wp_set_current_user( $original_user_id );
```

In normal operation, this is safe because the changeset's writer and capabilities were checked when the setting was created. The exploit does not use that normal write path; it fabricates the cached changeset object and its contents. The old safety argument no longer holds.

The stock `nav_menus_created_posts` setting is especially useful because it publishes selected draft posts with another sparse `wp_update_post()` call. That reaches the second hierarchy graph while WordPress's global current user is temporarily the original administrator. (Customizer user switch, navigation-setting save)

The strangest domino: a post transition fires `parse_request`

Every post save fires dynamic status/type hooks:

PHP Copy

```php
do_action(
    "{$new_status}_{$post->post_type}",
    $post->ID,
    $post,
    $old_status
);
```

Core also registers the REST loader on an existing action named `parse_request`:

PHP Copy

```php
add_action( 'parse_request', 'rest_api_loaded' );
```

The second hierarchy repair reaches **E**, whose cached status/type pair composes that exact hook name:

SEQUENCE Copy

```
new status: parse
post type:  request
hook name:  parse_request
```

This is a **dynamic-hook collision**. A post transition unexpectedly invokes a callback intended for the main HTTP request lifecycle. (Dynamic transition hook, default `parse_request` registration)

REST re-enters while the privileged identity is still active

In WordPress 7.0.1, `rest_api_loaded()` could call `serve_request()` even while the REST server was already dispatching the original batch.

The nested serving cycle inherits process-wide globals—including the current user temporarily selected by the Customizer. A protected request that returned `401` during the outer anonymous pass can now be evaluated with `create_users` capability and return `201`.

That is the authorization transition.

WordPress 7.0.2 closes it in two places. The REST loader returns early when a dispatch is active, and `serve_request()` refuses to start a fresh top-level cycle:

PHP Copy

```php
// WordPress 7.0.2 — simplified.
if ( $this->is_dispatching() ) {
    return false;
}
```

The REST re-entry patch is important defense in depth. WordPress did not rely on one signature: it repaired batch alignment, removed the SQL sink, and closed the nested-dispatch bridge.

The final step is intentionally ordinary

Once the attacker owns an administrator, WordPress is behaving as designed. An administrator with `install_plugins` can upload and activate PHP code when filesystem modifications are allowed.

In our isolated, stock WordPress 7.0.1 reproduction, the proof sequence was:

SEQUENCE Copy

```
anonymous protected user creation: 401
re-entered protected user creation: 201
normal administrator login:         confirmed
fixed execution canary:             /usr/bin/id
result:                             uid=33(www-data)
```

We used a fixed argument array, captured stdout, and removed the account, plugin, options, and owned rows afterward. There was no web shell or arbitrary command parameter.

`www-data` is not `root`, but this is still full application compromise. The PHP worker can usually read `wp-config.php`, access database credentials, modify writable site code, steal application secrets, impersonate users, and attack other resources reachable from the web tier.

The correct distinction is:

> The vulnerability is the anonymous path to administrator authority. Plugin execution is how that newly acquired authority becomes operating-system-level code execution.

Three fixes for three broken assumptions

Broken assumption	7.0.2 fix	Security effect
Every batch request index has a matching handler index	Add an error placeholder to `$matches`	Removes route confusion

Broken assumption	7.0.2 fix	Security effect
`author__not_in` is already an integer array	Always use `wp_parse_id_list()`	Removes the SQL injection sink
A top-level REST serving cycle cannot begin inside an active dispatch	Check `is_dispatching()` at both entry points	Removes privileged REST re-entry

That three-layer repair is good security engineering. If one patch is incomplete or a variant reaches a neighboring path, the other boundaries still interrupt the chain.

What about Redis or another persistent object cache?

Cloudflare reports that the RCE path applies when a persistent object cache is not in use. That matches the mechanism above: the two-request sequence relies on WordPress's default object cache disappearing between requests so the second request can populate those keys with forged objects. (Cloudflare analysis)

A persistent Redis or Memcached drop-in may cause this exact proof to fail because a genuine cached object survives into request two. That is an implementation detail, **not a supported mitigation**:

- The WordPress advisory does not exempt persistent-cache deployments.
- Drop-ins differ in invalidation, eviction, serialization, and multisite behavior.
- Cache configuration changes over time.
- A failed public or internal PoC does not prove that a vulnerable site is safe.

Patch the core. Do not deploy Redis as a security fix.

How to protect your sites now

1. Patch first

From a trusted host shell:

```bash
wp core version --extra
wp core check-update
wp core update
wp core update-db
wp core version --extra
wp core verify-checksums --include-root
```

If your deployment is intentionally pinned to a maintained branch, install at least **7.0.2**, **6.9.5**, or **6.8.6**, as appropriate. WordPress documents both the update process and `wp core update`.

For a specific package and locale:

```bash
wp core verify-checksums \
  --version=7.0.2 \
  --locale=en_US \
  --include-root
```

The official `wp core verify-checksums` command runs before WordPress loads, which is useful when active application code is not yet trusted.

Core checksums do not cover `wp-content`. Also review repository-hosted plugins:

```bash
wp plugin verify-checksums --all --strict
```

Private, premium, and custom plugins may not have WordPress.org checksums. “Checksum unavailable” is a prompt to compare with a trusted vendor artifact—not automatic proof of compromise. (Plugin checksum documentation)

2. If you cannot patch immediately, contain the batch endpoint

As a short-lived compensating control, deny anonymous `POST` requests to the normalized REST route `/batch/v1`. Cover both common WordPress route forms:

```
/wp-json/batch/v1
/?rest_route=/batch/v1
```

A vendor-neutral policy looks like this:

```
IF request.method == POST
AND normalized_wordpress_rest_route == "/batch/v1"
THEN block
```

Normalize encoded slashes and apply the rule at both the CDN and origin; a CDN rule does nothing if the origin is directly reachable. Test legitimate editor, plugin, and API workflows because batching is a supported feature.

Searchlight Cyber recommends this only as an emergency measure and warns that it can affect legitimate use. (Researcher mitigation guidance) Cloudflare has also deployed

managed rules for both CVEs, but stresses that WAF coverage does not replace patching. (Cloudflare WAF guidance)

3. Verify—do not trust the generator tag

The version in a page's HTML or an HTTP header can be hidden, cached, or rewritten. Verify the installed package through your hosting control plane, deployment inventory, or WP-CLI, then validate official checksums.

Forced automatic updates are excellent exposure reduction, not evidence that every site updated successfully.

Detection and hunting: look for the sequence, not one magic string

WordPress has not published universal attacker IPs, usernames, plugin names, hashes, or payload markers. A single `207 Multi-Status`, oEmbed row, Customizer changeset, or plugin installation is not proof of exploitation. All can be legitimate.

Look for correlation:

```
SEQUENCE                                                    Copy

A: anonymous POST to normalized /batch/v1 returns 207
B: a new administrator appears shortly afterward
C: a new login uses that account
D: plugin files or active_plugins change shortly afterward

A + B + C + D in a short window → critical investigation
```

Other useful leads include:

- Batch bodies containing an invalid or unparsable early `path` mixed with otherwise valid members.
- A scalar or string-shaped `author_exclude` value in a request that reaches a posts handler.
- PHP warnings involving an undefined or unexpected batch match index.
- Unusual clusters of `oembed_cache`, `customize_changeset`, and unknown post types.
- Cyclic `post_parent` relationships.
- A newly registered administrator immediately performing plugin upload or activation.
- New PHP files, child processes, or outbound connections from the web worker.

The privileged users operation may occur **inside the original PHP request**, so it may not appear as a separate `/wp/v2/users` line in an edge access log. Standard access logs also rarely include request bodies.

Start by locating batch requests in copied logs:

BASH Copy

```
rg -n -i \
  'POST .*?(wp-json/batch/v1|rest_route=(%2[fF]|/)batch(%2[fF]|/)v1)' \
  /path/to/copied-access-logs
```

Then inventory administrators and code from a trusted environment:

BASH Copy

```
wp user list \
  --role=administrator \
  --fields=ID,user_login,user_email,user_registered

wp plugin list --fields=name,status,version,update,auto_update
wp plugin list --status=must-use
```

```
wp plugin list --status=dropin

find wp-content/plugins wp-content/mu-plugins \
  -type f -name '*.php' -mtime -7 -print
```

Recent timestamps are leads, not verdicts. Legitimate deployments change files, and attackers can preserve or alter timestamps.

Database pivots

Replace `wp_` if the site uses another table prefix. Review administrator creation:

SQL Copy

```sql
SELECT
    u.ID,
    u.user_login,
    u.user_email,
    u.user_registered
FROM wp_users AS u
JOIN wp_usermeta AS m ON m.user_id = u.ID
WHERE m.meta_key = 'wp_capabilities'
  AND m.meta_value LIKE '%"administrator"%'
ORDER BY u.user_registered DESC;
```

Look for two-node parent cycles:

SQL Copy

```sql
SELECT
    child.ID,
    child.post_type,
    child.post_status,
    child.post_parent
FROM wp_posts AS child
JOIN wp_posts AS parent
  ON parent.ID = child.post_parent
```

```
WHERE child.ID <> 0
  AND parent.post_parent = child.ID;
```

A cycle can also result from a broken plugin or database corruption, and a capable attacker can remove evidence. Use it as an investigation pivot, not an automatic compromise finding.

If you suspect exploitation

Treat confirmed web-process RCE as a host incident, not merely a bad WordPress account.

1. **Restrict or isolate the site.** Stop further changes without destroying volatile evidence.
2. **Preserve first.** Snapshot the database, files, WAF events, CDN logs, origin access/error logs, PHP logs, process telemetry, and object-cache state before cleaning.
3. **Patch from a trusted source.** Verify core and plugin artifacts afterward.
4. **Review every identity.** Check administrators, application passwords, active sessions, hosting-panel accounts, deployment keys, SFTP/SSH users, and database users.
5. **Review every execution surface.** Inspect plugins, themes, MU plugins, drop-ins, uploads, cron jobs, `.htaccess`, `.user.ini`, `wp-config.php`, temporary directories, and server-level persistence.
6. **Revoke sessions and rotate secrets after evidence collection.** WordPress's official hacked-site guidance recommends resetting access and replacing secret keys. (WordPress hacked-site guide)
7. **Rebuild from known-good artifacts if execution is confirmed.** Deleting the visible administrator or plugin is not sufficient.

8. **Expand the blast-radius review.** Investigate other sites sharing the same Unix user, document root, database account, control panel, or secrets.

Useful session and salt rotation commands include:

```bash
wp user list --field=ID \
  | xargs -n 1 wp user session destroy --all

wp config shuffle-salts
```

These commands are documented by WordPress, but run them only after preserving evidence. (Session destruction, salt rotation)

Five engineering lessons worth keeping

1. Parallel arrays are a security boundary when indexes carry identity

If several arrays describe the same logical objects, every error path must preserve cardinality and alignment. Better yet, store one structured record per request.

2. Validate at the sink, even when an upstream schema promises safety

The REST schema said “array of integers.” `WP_Query` still needed to normalize every input shape before constructing SQL. Defense in depth matters because dispatchers, hooks, filters, and internal callers can bypass the expected route.

3. Caches can be trust boundaries

`get_post()` does not mean “read this row from the database.” It means “obtain the current object,” possibly from memory. Code that merges cached objects into writes must consider how those objects were populated.

4. Global identity plus re-entrancy is combustible

Temporarily changing a process-wide current user can be safe only when no unrelated work can re-enter during that window. Nested dispatch turned a short-lived internal identity into authority for a new external operation.

5. Dynamic hook names can create surprising control-flow edges

`{$status}_{$type}` looks harmless until attacker-influenced values compose the name of an existing lifecycle hook. Dynamic dispatch deserves the same threat modeling as an indirect function call.

Frequently asked questions

Is this genuinely unauthenticated?

Yes. The initial chain begins at the public REST batch endpoint and requires no existing WordPress login. The official advisory describes it as a REST batch-route confusion combined with SQL injection leading to RCE.

Does the RCE work without the SQL injection?

The official record says the opposite: **route confusion combined with the SQL injection** leads to RCE. Claims that the stock chain needs no SQLi are not supported by the WordPress advisory.

Does it require a vulnerable plugin?

No. Searchlight Cyber states that the issue is exploitable on a stock installation with no plugins. The plugin appears only at the end, after administrator access is obtained, as a normal mechanism for executing PHP. (Searchlight Cyber research notice)

Is `uid=33(www-data)` the same as root?

No. It is the operating-system identity of the PHP/web-server worker in many Linux deployments. It is still enough for full WordPress compromise and may expose adjacent secrets and services.

Is a WAF enough?

No. It can reduce exposure while the update rolls out, but it does not repair vulnerable core code. Encodings, alternate routing, origin bypass, and future variants make signature-only defenses fragile.

Can we assume forced automatic updates protected us?

No. Verify the installed version and checksums. Updates can fail because of filesystem permissions, disabled file modifications, maintenance locks, custom deployment systems, or connectivity problems.

Final thought

The striking part of this chain is not that WordPress exposed one obviously catastrophic function. It is that several ordinary-looking assumptions amplified one another:

- an error path broke index alignment;

- a numeric parameter reached SQL without universal normalization;
- query results became trusted cached objects;
- maintenance code persisted those objects;
- a temporary user switch overlapped a nested dispatch;
- a normal administrator feature supplied the final code execution.

Patch now—but keep the engineering lesson.

Security boundaries often fail in the seams between components, especially when one layer assumes the previous layer kept its bookkeeping straight.

Primary references

- WordPress 7.0.2 release announcement
- CVE-2026-63030 / GHSA-ff9f-jf42-662q
- CVE-2026-60137 / GHSA-fpp7-x2x2-2mjf
- Batch-alignment patch
- `author__not_in` normalization patch
- REST re-entry patch
- Searchlight Cyber's wp2shell notice
- Cloudflare's WAF and cache-condition guidance

ABOUT

BugBunny is a fully autonomous offensive security platform that conducts comprehensive penetration testing without human intervention.

PRODUCT

- Audit Console
- Dashboard
- Pricing
- Referral Program
- How It Works

COMPARE

- XBOW Alternative
- Aikido Alternative
- Novee Alternative
- Penligent Alternative
- Strix Alternative

RESEARCH

- Blog
- Hall of Fame
- Disclosure

CONNECT

- Email
- Sales

SOCIAL

- Twitter
- GitHub

DIAGNOSTICS

PLATFORM	Linux aarch64	LANGUAGE	ru-RU
CORES	8	MEMORY	8GB
VIEWPORT	374×649	SCREEN	339×735
COLOR DEPTH	24BIT	PIXEL RATIO	3.1875
TIMEZONE	Europe/Moscow	COOKIES	ENABLED
NETWORK	4G	LOCAL STORAGE	AVAILABLE
SESSION STORAGE	AVAILABLE		
UTC	05:46:59	LOCAL	08:46:59
UNIX	1785995219	UPTIME	6S
HOST			bugbunny.ai
STAT			ONLINE
YOUR IP			2.26.93.50
LOCATION			Stockholm, Stockholm, Sweden

USER AGENT Mozilla/5.0 (Linux; arm_64; Android 16; SM-S938B) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.7778.58 YaBrowser/26.6.6.58.00 Mobile Safari/537.36